

라이브 서비스 게임 패치 지원 플랫폼 개발계획서

1. 프로젝트 목적

라이브 서비스 게임 패치 지원 플랫폼을 만들고자 함

- Unity Engine의 무배포 패치 구현을 돕는 표준 라이브러리인 Addressable, AssetBundle에 적합한 백엔드를 구현할 계획임
- 파일 서버 뿐만이 아니라, 게임 개발 팀의 배포 파이프라인을 지원하기 위한 여러 기능들을 구현할 계획임

본 프로젝트의 기대 효과는 다음과 같음

- 게임 개발자가 클라이언트 개발에만 집중할 수 있게 함
- 백엔드 인프라에 대한 이해도가 없는 게임 개발 팀의 지속적 패치, 게임 버전 관리를 지원함

2. 주요 기능

개발자(Developer)

- 개발자를 등록할 수 있음
- 개발자를 등록하면 개발자와 연결된 루트 계정이 생성됨
- 루트 계정 또는 Admin(개발자 권한) 권한을 가진 계정은 연결된 개발자의 정보를 수정할 수 있음
- 루트 계정은 연결된 개발자를 삭제할 수 있음

게임(Game)

- 루트 계정 또는 Admin(개발자 권한), Publisher 권한을 가진 계정은 연결된 개발자의 게임을 등록할 수 있음
- 루트 계정 또는 Admin(개발자 권한), Publisher 권한을 가진 계정은 연결된 개발자의 게임의 정보를 수정할 수 있음
- 루트 계정 또는 Admin(개발자 권한) 권한을 가진 계정은 연결된 개발자의 게임을 삭제할 수 있음

계정(Account)

- 루트 계정 또는 Admin(개발자 권한) 권한을 가진 계정은 개발자의 게임과 연결된 계정을 생성할 수 있음
- 루트 계정 또는 Admin(개발자 권한) 권한을 가진 계정은 개발자의 게임과 연결된 계정을 관리할 수 있음
- 루트 계정 또는 Admin(게임 권한) 권한을 가진 계정은 연결된 게임과 연결된 계정을 생성할 수 있음
- 루트 계정 또는 Admin(게임 권한) 권한을 가진 계정은 연결된 게임과 연결된 계정을 관리할 수 있음

패치(Patch)

- 루트 계정 또는 Admin(개발자 권한), Admin(게임 권한), Publisher, Maintain 권한을 가진 계정은 접근 가능한 게임의 패치를 업로드할 수 있음
- 루트 계정 또는 Admin(개발자 권한), Admin(게임 권한), Publisher, Maintain 권한을 가진 계정은 접근 가능한 게임의 패치의 패치 노트를 수정할 수 있음
- 패치에는 패치 노트, 플랫폼 별 패치 파일 등이 포함됨
- 모든 클라이언트는 API를 통해 게임의 특정 플랫폼 및 버전의 패치 파일을 다운로드받을 수 있음
 - 파일은 로컬에 저장되도록 구현할 예정이며, 프로젝트에 적합하지 않으면 다른 방식(예: 클라우드으로 구현될 수도 있음)

권한 (Permission)

- 계정은 여러 개의 개발자 권한 및 게임 권한(게임 별 권한)을 가질 수 있음
- 계정은 하나의 권한에서만 특정 기능에 대한 접근이 허가되면 다른 권한의 설정에 관계 없이 접근할 수 있음
- 권한의 종류 및 권한 별 허가 기능은 다음과 같음

	루트 계정 권한	Admin(개발자)	Publisher(개발자)	Admin(게임)	Primary Write(게임)	Maintain(게임)
개발자 정보 수정	O	O	X	X	X	X
개발자 삭제	O	X	X	X	X	X
게임 등록	O	O	O	X	X	X
게임 정보 수정	O	O	O	O	O	X
게임 삭제	O	O	X	X	X	X
계정 생성	O	O	X	X	X	X
계정 관리	O	O	X	X	X	X
게임 별 계정 생성	O	O	X	O	X	X
게임 별 계정 관리	O	O	X	O	X	X
게임 패치 배포	O	O	X	O	X	O

인증(Authentication)

- 계정은 조직 식별자, 개인 식별자, 비밀번호를 통해 인증할 수 있음
- 인증 정보는 클라이언트에 토큰 형태로 저장됨

3. 기술 스택 및 개발 환경

1. Spring Boot
2. Spring Security
3. JPA — ORM, JPA Repository, Auto DDL 등의 기능을 통해 빠른 개발이 가능하여 선정
4. MySQL — 수행평가 기본은 h2 DB이나, 파일 서버의 특징에 따라 영속적 데이터 저장 기능이 필요했으므로 선정
5. JWT — JWT 기반 인증 구현을 위한 편의성 API를 제공하므로 선정
6. Apache Commons FileUpload — 파일 업로드 기능 구현 과정에서 Spring 기본 Multipart의 한계점이 드러나 Multipart 대신 사용한 스트리밍 지원 솔루션

4. 프로젝트 설계

시스템 아키텍처

- Presentation
 - Controller 및 Request, Response DTO 포함
 - Application 계층과 도메인 구성을 공유함
- Application
 - 하나의 내부적 행위를 제공하는 Service와 Service를 조합하여 기능 별 흐름을 만드는 UseCase가 포함됨
 - 외부에 기술적 요구를 하는 Outbound Port(인터페이스)가 포함됨

- Domain
 - Entity가 포함됨
 - Entity는 외부 수정을 막고 일부 메서드만 공개하여 데이터의 의미를 지킴
- Infrastructure
 - Application 계층에서 정의된 Outbound Port를 실제 기술을 통해 구현함

도메인 설계

- 테이블
 - account
 - developer
 - filemeta
 - game
 - refreshtoken
 - gameaccesspermission
 - developeraccesspermission
 - patch
 - 등등
- 도메인
 - account
 - auth
 - developer
 - file
 - permission
 - patch
 - 등등

5. 개발 일정

1주차

- 시스템 아키텍처 설계
- 도메인 설계
- 테이블 설계
- Spring WebMVC 기반 기본 CRUD 구현

2주차

- 인증 기능 구현
- (중요)파일 서버 기능 구현

3주차

- 권한 기능 구현

4주차

- API 사양 설계(실제 게임 엔진 맞춤형)
- API 구현

5주차(~05.28.)

- 테스트 및 오류 수정